

Fundamentos em Telecomunicações em Tecnologias Facilitadoras

- Fundamentos de infraestrutura de TI
- Tema principal dessa apresentação: contêineres e microserviços, mostrando como essas tecnologias resolvem problemas de implantação, isolamento, escalabilidade e operação de aplicações modernas, principalmente no contexto de telecomunicações e 5G

Contêineres e microserviços

- Contêineres e microserviços são conceitos muito importantes na infraestrutura moderna de TI
- Eles aparecem como uma evolução natural depois de infraestrutura física, máquinas virtuais, virtualização de recursos, cloud e sistemas ICT
- A ideia principal é tornar as aplicações mais leves, modulares, portáteis, escaláveis e fáceis de gerenciar
- Em telecomunicações, esses conceitos se tornaram ainda mais importantes com a chegada do 5G
- Isso acontece porque redes 5G exigem serviços com baixa latência, alta disponibilidade, grande escala e capacidade de adaptação rápida

Problema inicial

- Nos módulos anteriores foi visto que tanto máquinas físicas quanto máquinas virtuais executam aplicações sobre um sistema operacional
- O sistema operacional é necessário porque fornece os recursos básicos para a aplicação funcionar
- Ele gerencia processos, memória, arquivos, rede, permissões e recursos do hardware
- Porém, executar muitas aplicações no mesmo sistema operacional pode gerar problemas
- Cada aplicação pode precisar de bibliotecas, versões, dependências e configurações diferentes
- Quando várias aplicações compartilham o mesmo ambiente, uma dependência pode conflitar com outra

Processos no sistema operacional

- Desde sua concepção, os sistemas operacionais executam muitos processos independentes
- Cada aplicação pode gerar vários processos

- Em ambientes corporativos, um mesmo servidor pode executar diversos serviços ao mesmo tempo
- O problema é que esses serviços podem ter requisitos diferentes
- Exemplos:
 - Uma aplicação precisa de uma versão específica de biblioteca
 - Outra aplicação precisa de outra versão incompatível
 - Um serviço exige configuração específica de rede
 - Outro serviço exige permissões diferentes
 - Uma aplicação pode consumir recursos demais e prejudicar outras
- Esse cenário cria dificuldade de manutenção, implantação e estabilidade

Dependências

- Dependências são recursos que uma aplicação precisa para funcionar
- Podem ser bibliotecas, frameworks, arquivos de configuração, versões de linguagem, pacotes e serviços auxiliares
- Exemplo:
 - Uma aplicação Python pode depender de uma versão específica do TensorFlow
 - Outra pode depender de uma versão diferente
 - Se as duas forem instaladas no mesmo sistema, pode haver conflito
- Contêineres ajudam a resolver isso porque empacotam a aplicação junto com suas dependências

O que são contêineres?

- Contêineres são ambientes isolados usados para executar aplicações
- Eles empacotam a aplicação e tudo que ela precisa para funcionar
- Isso inclui bibliotecas, dependências, arquivos e configurações
- O contêiner roda sobre o sistema operacional, mas isola a aplicação do restante do ambiente
- A ideia é que a aplicação funcione da mesma forma em diferentes lugares
- Exemplo:
 - Funciona no computador do desenvolvedor
 - Funciona no servidor de teste
 - Funciona no ambiente de produção
 - Funciona na cloud

Por que contêineres foram criados?

- A tecnologia de contêineres ajuda a resolver problemas associados ao isolamento de aplicações
- Também reduz o chamado overhead
- Overhead é o custo extra de processamento, memória ou armazenamento necessário para executar uma solução
- Uma máquina virtual precisa carregar um sistema operacional completo
- Um contêiner não precisa carregar um sistema operacional completo separado
- Por isso, contêineres são mais leves que máquinas virtuais
- Eles permitem executar aplicações isoladas com menor consumo de recursos

Contêineres x máquinas virtuais

Conceito	Máquina virtual	Contêiner
Isolamento	Executa um sistema operacional completo	Isola processos e dependências da aplicação
Peso	Mais pesada	Mais leve
Inicialização	Mais lenta	Mais rápida
Uso de recursos	Consome mais CPU, memória e disco	Consome menos recursos
Portabilidade	Boa	Muito alta
Uso típico	Ambientes completos e isolados	Aplicações, serviços e microserviços

Como funciona um contêiner?

- Um contêiner usa o kernel do sistema operacional do host
- Kernel é o núcleo do sistema operacional
- Ele gerencia recursos como CPU, memória, processos e dispositivos
- O contêiner cria um ambiente isolado para a aplicação
- Dentro desse ambiente ficam a aplicação e suas dependências
- O contêiner não precisa instalar um sistema operacional inteiro dentro dele
- Isso torna a execução mais rápida e eficiente

Imagem de contêiner

- Uma imagem de contêiner é um pacote com tudo que a aplicação precisa para executar
- Ela é como um molde
- A partir da imagem, é possível criar um ou vários contêineres
- Exemplo:

- Uma imagem contém a aplicação, bibliotecas e configurações
- Ao executar essa imagem, nasce um contêiner
- A imagem garante que o ambiente seja reproduzível
- Isso reduz problemas de compatibilidade

Contêiner em execução

- Um contêiner em execução é uma instância de uma imagem
- Se a imagem é o molde, o contêiner é a aplicação rodando de fato
- É possível ter vários contêineres criados a partir da mesma imagem
- Isso facilita escalabilidade
- Se a demanda aumenta, novos contêineres podem ser criados
- Se a demanda diminui, contêineres podem ser removidos

Vantagens dos contêineres

- Portabilidade
- Isolamento
- Inicialização rápida
- Menor consumo de recursos
- Facilidade de implantação
- Padronização do ambiente
- Escalabilidade
- Melhor uso da infraestrutura
- Facilidade para automação
- Boa integração com cloud e DevOps

Portabilidade

- Portabilidade significa que a aplicação pode ser executada em diferentes ambientes com menos problemas
- Como o contêiner leva as dependências junto, o ambiente fica mais previsível
- Isso reduz o problema clássico:
 - “Na minha máquina funciona”
- O mesmo contêiner pode rodar em:
 - Notebook do desenvolvedor
 - Servidor local
 - Data center
 - Cloud pública

- Cloud privada
- Ambiente de telecomunicações

Isolamento

- Isolamento significa que uma aplicação fica separada das outras
- Cada contêiner possui seu próprio ambiente de execução
- Se uma aplicação tiver conflito de dependência, isso não deve afetar as demais
- Se um contêiner falhar, os outros podem continuar funcionando
- Esse isolamento melhora estabilidade e segurança

Menor overhead

- Contêineres têm menor overhead que máquinas virtuais porque não carregam um sistema operacional completo para cada aplicação
- Eles compartilham o kernel do sistema operacional
- Isso permite executar mais aplicações com menos recursos
- Em ambientes de grande escala, essa diferença é muito importante
- Telecomunicações e cloud precisam lidar com muitos serviços ao mesmo tempo
- Por isso, contêineres são úteis para melhorar eficiência

Docker

- Docker é uma das tecnologias mais conhecidas para criação e execução de contêineres
- Ele popularizou o uso de contêineres em desenvolvimento e produção
- Com Docker, é possível criar imagens, executar contêineres e distribuir aplicações de forma padronizada
- Ele facilita a implantação porque empacota a aplicação com suas dependências
- Em ambientes modernos, Docker costuma aparecer junto com Kubernetes

Kubernetes

- Kubernetes é uma plataforma de orquestração de contêineres
- Orquestrar significa gerenciar automaticamente a execução, escala, recuperação e implantação dos contêineres
- Quando há poucos contêineres, é possível gerenciar manualmente
- Mas em ambientes grandes, como telecomunicações e cloud, isso se torna inviável
- Kubernetes ajuda a controlar muitos contêineres ao mesmo tempo

Funções do Kubernetes

- Kubernetes pode:
 - Criar contêineres
 - Reiniciar contêineres com falha
 - Escalar aplicações
 - Distribuir carga entre instâncias
 - Atualizar aplicações
 - Gerenciar configurações
 - Controlar recursos
 - Ajudar na alta disponibilidade
- Em redes 5G e ambientes cloud-native, Kubernetes se torna essencial para operação automatizada

Microserviços

- Microserviço é uma forma de estruturar uma aplicação como um conjunto de serviços pequenos e independentes
- Cada microserviço executa uma função específica
- Os microserviços se comunicam entre si por meio de APIs
- API significa Application Programming Interface
- Em português, interface de programação de aplicações
- A API permite que um serviço converse com outro de forma padronizada

Ideia principal dos microserviços

- Em vez de construir uma aplicação enorme e única, a aplicação é dividida em partes menores
- Cada parte executa uma responsabilidade
- Exemplo de uma aplicação de e-commerce:
 - Serviço de login
 - Serviço de catálogo
 - Serviço de pagamento
 - Serviço de carrinho
 - Serviço de entrega
 - Serviço de notificação
- Cada serviço pode ser desenvolvido, implantado e escalado separadamente

Arquitetura monolítica

- A arquitetura monolítica é o modelo em que todos os componentes da aplicação ficam juntos em uma única estrutura

- Nesse modelo, as partes da aplicação são interdependentes
- Uma mudança em uma parte pode afetar o sistema inteiro
- O monólito pode ser mais simples no início
- Porém, conforme o sistema cresce, ele pode se tornar difícil de manter, escalar e atualizar

Problemas da arquitetura monolítica

- Dificuldade de escalar apenas uma parte da aplicação
- Dependência forte entre componentes
- Atualizações mais arriscadas
- Maior dificuldade para equipes trabalharem de forma independente
- Menor flexibilidade para usar tecnologias diferentes
- Uma falha pode afetar o sistema inteiro
- Deploys podem ser lentos e complexos

Microserviços como alternativa ao monólito

- A arquitetura de microserviços surge como alternativa ao modelo monolítico
- A aplicação é dividida em serviços pequenos e independentes
- Cada microserviço pode evoluir no seu próprio ritmo
- Isso melhora:
 - Escalabilidade
 - Desenvolvimento independente
 - Resiliência
 - Flexibilidade tecnológica
 - Manutenção
 - Implantação

Desenvolvimento independente

- Em microserviços, equipes diferentes podem trabalhar em serviços diferentes
- Uma equipe pode desenvolver o serviço de autenticação
- Outra pode desenvolver o serviço de pagamento
- Outra pode cuidar do serviço de notificações
- Isso acelera o desenvolvimento
- Também reduz o risco de uma alteração pequena exigir mudanças no sistema inteiro

Implantação independente

- Microserviços podem ser implantados separadamente

- Isso significa que atualizar um serviço não exige atualizar toda a aplicação
- Exemplo:
 - O serviço de pagamento pode ser atualizado sem alterar o serviço de login
 - O serviço de notificação pode ser reiniciado sem derrubar o sistema inteiro
- Essa independência melhora agilidade e reduz risco operacional

Escalabilidade em microserviços

- Microserviços permitem escalar apenas a parte que precisa de mais recurso
- Exemplo:
 - Em uma loja online, o serviço de pagamento pode ter alta demanda em uma promoção
 - O serviço de pagamento pode ser escalado sem aumentar todos os outros serviços
- Isso é mais eficiente do que escalar uma aplicação monolítica inteira
- Em telecomunicações, essa lógica é importante porque diferentes funções da rede podem ter demandas diferentes

Resiliência em microserviços

- Resiliência é a capacidade de continuar funcionando mesmo diante de falhas
- Em microserviços, se um serviço falhar, o sistema pode continuar parcialmente funcionando
- Exemplo:
 - O serviço de recomendação falha
 - O usuário ainda consegue acessar catálogo e comprar
- Para isso funcionar bem, é necessário projetar comunicação, monitoramento e tratamento de falhas corretamente

Tecnologia heterogênea

- Microserviços permitem usar tecnologias diferentes em partes diferentes da aplicação
- Um serviço pode ser escrito em Java
- Outro pode ser escrito em Python
- Outro pode usar Node.js
- Outro pode usar Go
- O importante é que todos consigam se comunicar por APIs
- Isso dá liberdade tecnológica, mas também exige bom gerenciamento

APIs

- APIs são interfaces que permitem comunicação entre sistemas ou serviços
- Em microserviços, APIs são essenciais porque os serviços precisam trocar informações
- Uma API define como um serviço pode ser acessado
- Exemplo:
 - O serviço de pedidos chama a API do serviço de pagamento
 - O serviço de pagamento retorna aprovado ou recusado
- APIs bem definidas ajudam a manter independência entre microserviços

Microserviços e contêineres juntos

- Microserviços e contêineres combinam muito bem
- Cada microserviço pode ser empacotado em um contêiner
- Isso facilita implantar, escalar e atualizar cada serviço separadamente
- Kubernetes pode orquestrar todos esses contêineres
- A combinação fica assim:
 - Microserviço define a arquitetura da aplicação
 - Contêiner empacota e executa cada serviço
 - Kubernetes gerencia os contêineres em escala

Relação entre contêineres, microserviços e Kubernetes

Conceito	Função
Microserviço	Divide a aplicação em serviços pequenos e independentes
Contêiner	Empacota e executa cada serviço com suas dependências
Docker	Ferramenta usada para criar e executar contêineres
Kubernetes	Orquestra contêineres em larga escala
API	Permite comunicação entre microserviços

Contêineres e microserviços em telecomunicações

- Desde a implementação do 5G, contêineres e microserviços se tornaram termos comuns nas equipes de implementação, operação e manutenção de redes
- Isso acontece porque telecomunicações exigem serviços escaláveis, flexíveis e de alta disponibilidade
- As redes modernas precisam lidar com:
 - Muitos usuários simultâneos
 - Grandes volumes de dados

- Serviços em tempo real
- Baixa latência
- Alta confiabilidade
- Qualidade de serviço
- Dispositivos IoT
- Funções de rede distribuídas

QoS

- QoS significa Quality of Service
- Em português, qualidade de serviço
- QoS envolve garantir que determinados serviços recebam desempenho adequado
- Isso pode incluir:
 - Baixa latência
 - Largura de banda mínima
 - Prioridade de tráfego
 - Baixa perda de pacotes
 - Alta disponibilidade
- Em telecomunicações, QoS é fundamental porque diferentes aplicações têm requisitos diferentes

Desafios atuais das telecomunicações

- O cenário atual de telecomunicações enfrenta desafios específicos
- Principais desafios:
 - Gerenciar grandes volumes de dados
 - Fornecer serviços em tempo real
 - Garantir QoS para milhões de usuários
 - Operar redes distribuídas
 - Atualizar serviços sem interrupção
 - Lidar com grande quantidade de dispositivos conectados
 - Manter alta disponibilidade
 - Reduzir latência
- Contêineres e microserviços ajudam a enfrentar esses desafios

Como contêineres ajudam telecomunicações?

- Permitem implantação rápida de funções e serviços
- Reduzem overhead em comparação com máquinas virtuais

- Facilitam portabilidade entre ambientes
- Melhoram padronização
- Facilitam automação
- Permitem escalar serviços conforme a demanda
- Ajudam a criar ambientes mais leves e flexíveis
- São adequados para cloud, edge e data centers distribuídos

Como microserviços ajudam telecomunicações?

- Dividem sistemas complexos em partes menores
- Permitem atualizar uma função sem derrubar o sistema inteiro
- Facilitam escalabilidade por serviço
- Melhoram resiliência
- Permitem desenvolvimento independente
- Ajudam a construir aplicações cloud-native
- Permitem adaptar rapidamente serviços a novas demandas

5G e cloud-native

- O 5G é fortemente associado a arquiteturas cloud-native
- Cloud-native significa projetar sistemas para rodar em ambientes de cloud, com automação, escalabilidade e resiliência
- Contêineres, microserviços e Kubernetes são bases importantes desse modelo
- Em vez de depender apenas de equipamentos físicos dedicados, funções de rede podem rodar como software em ambientes distribuídos
- Isso ajuda a tornar a rede mais flexível e escalável

Funções de rede em contêineres

- Em redes modernas, funções de rede podem ser implementadas como software
- Quando essas funções são empacotadas em contêineres, elas podem ser implantadas com mais agilidade
- Exemplos de funções ou serviços que podem usar esse modelo:
 - Funções do núcleo 5G
 - Gateways
 - Serviços de autenticação
 - Monitoramento
 - Funções de controle
 - Aplicações de borda

- Esse modelo é importante para NFV e arquiteturas cloud-native

Relação com edge computing

- Edge computing significa processar dados mais perto do usuário ou dispositivo
- Contêineres são úteis no edge porque são leves e rápidos de implantar
- Microserviços ajudam a dividir funções que podem ser executadas em pontos diferentes da rede
- Em 5G, edge computing é importante para aplicações de baixa latência
- Exemplos:
 - Carros conectados
 - Indústria 4.0
 - Realidade aumentada
 - Monitoramento em tempo real
 - Telemedicina

Alta disponibilidade

- Alta disponibilidade significa manter o serviço funcionando mesmo quando há falhas
- Contêineres e microserviços ajudam nisso porque permitem replicar serviços e reiniciar componentes automaticamente
- Kubernetes pode detectar falhas e recriar contêineres
- Se uma instância falha, outra pode continuar atendendo
- Isso é essencial em telecomunicações, porque a rede precisa operar continuamente

Escalabilidade

- Escalabilidade é a capacidade de aumentar ou reduzir recursos conforme a demanda
- Em contêineres e microserviços, isso pode ser feito por serviço
- Exemplo:
 - Um microserviço com alta demanda pode ser replicado em vários contêineres
 - Outro microserviço com pouca demanda pode permanecer com menos instâncias
- Isso melhora o uso de recursos e reduz custos

Agilidade operacional

- Contêineres e microserviços aumentam a agilidade operacional
- Novas versões podem ser implantadas mais rápido
- Correções podem ser feitas em serviços específicos
- Equipes podem trabalhar em paralelo

- Ambientes podem ser recriados automaticamente
- Isso reduz tempo de implantação e facilita inovação

Observabilidade

- Observabilidade é a capacidade de entender o que está acontecendo no sistema a partir de métricas, logs e rastreamentos
- Em microserviços, isso é muito importante porque há muitos serviços se comunicando
- Para operar bem, é necessário monitorar:
 - Desempenho
 - Erros
 - Latência
 - Consumo de recursos
 - Comunicação entre serviços
- Sem observabilidade, fica difícil identificar onde está a falha

Desafios de contêineres e microserviços

- Apesar das vantagens, esses modelos também trazem desafios
- Principais desafios:
 - Maior complexidade operacional
 - Necessidade de orquestração
 - Monitoramento mais complexo
 - Segurança entre serviços
 - Gerenciamento de APIs
 - Controle de versões
 - Comunicação entre microserviços
 - Dependência de automação
- Por isso, ferramentas como Kubernetes, observabilidade e políticas de segurança são fundamentais

Segurança em contêineres

- Contêineres também precisam de segurança
- Cuidados importantes:
 - Usar imagens confiáveis
 - Atualizar dependências
 - Evitar permissões excessivas
 - Escanear vulnerabilidades

- Isolar redes
- Controlar segredos e credenciais
- Monitorar execução
- Em telecomunicações, segurança é ainda mais crítica porque os serviços podem fazer parte de infraestrutura essencial

Resumo

- Contêineres:
 - Empacotam aplicação e dependências
 - Isolam a aplicação do ambiente
 - São mais leves que máquinas virtuais
 - Reduzem overhead
 - Facilitam portabilidade e implantação
- Docker:
 - Tecnologia muito usada para criar e executar contêineres
- Kubernetes:
 - Orquestra contêineres em larga escala
 - Ajuda em escalabilidade, recuperação e automação
- Microserviços:
 - Dividem uma aplicação em serviços pequenos e independentes
 - Cada serviço executa uma função específica
 - A comunicação ocorre por APIs
- Arquitetura monolítica:
 - Todos os componentes ficam interdependentes
 - Pode dificultar escalabilidade, manutenção e atualizações
- Microserviços resolvem:
 - Escalabilidade
 - Desenvolvimento independente
 - Resiliência
 - Uso de tecnologias diferentes
- Contêineres + microserviços:
 - Cada microserviço pode rodar em um contêiner
 - Isso facilita implantação, escala e operação
- Telecomunicações:
 - Contêineres e microserviços são importantes desde a implementação do 5G
 - Ajudam a lidar com grande volume de dados, serviços em tempo real, QoS, alta disponibilidade e muitos dispositivos conectados

Qual foi o principal motivador para a utilização de contêineres em cenários de virtualização?

- ☐ Facilidade de gerenciamento de múltiplos sistemas operacionais ao mesmo tempo.
- ☒ Problemas relacionados à execução de várias aplicações no mesmo sistema operacional, devido a requisitos específicos.
- ☐ Necessidade de isolamento entre diferentes sistemas operacionais.
- ☐ Redução do número de bibliotecas utilizadas por cada aplicação.

Sua pontuação: 10 de 40

Questão 1 de 4

Continuar >

Qual é a vantagem principal do uso de contêineres em comparação com máquinas virtuais?

- ☐ Maior segurança por meio da execução de múltiplos sistemas operacionais.
- ☐ Eliminação completa do overhead associado ao uso de recursos.
- ☒ Melhor desempenho devido ao compartilhamento do kernel do sistema operacional.
- ☐ Capacidade de executar múltiplas máquinas físicas em um único contêiner.

Qual é um dos principais fatores que garantem a portabilidade dos contêineres?

- ☐ Execução de múltiplos sistemas operacionais diferentes em cada contêiner.
- ☐ Isolamento total dos recursos físicos do sistema.
- ☐ Capacidade de encapsular o kernel do sistema operacional dentro do contêiner.
- ☒ Inclusão de todos os componentes necessários para a aplicação, permitindo que ela rode em qualquer infraestrutura compatível.

Qual é uma das principais funções do Kubernetes no gerenciamento de contêineres?

- ☒ Gerenciamento de contêineres em clusters e balanceamento de carga entre eles.
- ☐ Padronização do empacotamento de aplicações e dependências.
- ☐ Automação da criação de imagens de contêineres para diferentes ambientes.
- ☐ Combinação de múltiplos diretórios em um único diretório para contêineres.